On Computing the Hyperparameter of Extreme Learning Machines: Algorithm and Application to Computational PDEs

Summary of Dong and Yang (2021)

1 Introduction

Extreme Learning Machine (ELM) is a neural network-based method for solving computational partial differential equations (PDEs) that has shown promising results. The key characteristics of ELM are:

- Random but fixed (non-trainable) hidden-layer coefficients
- Trainable linear output-layer coefficients determined by a linear or nonlinear least squares method

This approach differs from traditional deep neural network (DNN)-based PDE solvers which typically adjust all network weights through gradient descent optimization. While DNN-based methods face challenges with limited accuracy, lack of consistent convergence rates, and high computational costs, ELM methods have demonstrated exponential convergence behavior and high accuracy.

A critical hyperparameter in ELM is R_m, the maximum magnitude of random coefficients used to initialize the hidden layers. The choice of R_m significantly affects solution accuracy, with optimal results typically achieved with moderate R_m values.

2 ELM Algorithm for Solving PDEs

2.1 Basic ELM Architecture

Consider a domain Ω in d dimensions and a PDE system:

$$L(u) = f(x) \tag{1}$$

$$B(u) = g(x) \text{ on } \partial\Omega \tag{2}$$

where L and B are differential operators (potentially nonlinear), $u(x)$ is the solution to be determined, and $f(x)$ and $g(x)$ are given functions.

The solution is approximated using a feed-forward neural network with architecture $[M_0, M_1, ..., M_L]$, where:

- $M_0 = d$ (input dimension)
- $M_L =$ dimension of u (output dimension)
- $L + 1$ is the total number of layers
- M_i is the number of nodes in layer i

For a set of collocation points $Z = \{x_i \in \Omega, 1 \leq i \leq Q\}$ with boundary points $Z_b = \{x_i \in Z \text{ and } x_i \in \partial\Omega, 1 \leq i \leq Q_b\}$, the neural network is defined by:

$$\beta^l = \sigma(\beta^{l-1} W^l + b^l), \quad 1 \leq l \leq L-1 \tag{3}$$

$$U = \beta^{L-1} W^L \tag{4}$$

where $\sigma(\cdot)$ is the activation function, W^l is the weight matrix for layer l, and b^l is the bias vector.

2.2 Single-R_m-ELM Configuration

In the traditional ELM implementation (Single-R_m-ELM), all hidden layer weights and biases (W^l, b^l) for $1 \leq l \leq L-1$ are set to random values generated uniformly on $[-R_m, R_m]$ and fixed. Only the output layer weights W^L are trainable.

The solution on the continuum level is expressed as:

$$u_i(x) = \sum_{j=1}^{M} V_j(x) \omega_{ji}, \quad 1 \leq i \leq M_L \tag{5}$$

where $V_j(x)$ are the output fields of the last hidden layer, and ω_{ji} are the trainable output layer weights.

2.3 Multi-R_m-ELM Configuration

A key innovation in the paper is the Multi-R_m-ELM configuration, where each hidden layer uses a different R_m value:

- Hidden layer l uses weights/biases randomly generated on $[-R_m^{(l)}, R_m^{(l)}]$
- The set of hyperparameters is represented as $\mathbf{R_m} = (R_m^{(1)}, R_m^{(2)}, ..., R_m^{(L-1)})$

This configuration generally provides higher accuracy than Single-R_m-ELM.

2.4 Training Process

For a given PDE system, training proceeds as follows:

1. Substitute the ELM representation into the PDE:

$$L\left(\sum_{j=1}^{M} V_j(x_p)\omega_j\right) = f(x_p), \quad \forall x_p \in Z, 1 \leq p \leq Q \tag{6}$$

$$B\left(\sum_{j=1}^{M} V_j(x_q)\omega_j\right) = g(x_q), \quad \forall x_q \in Z_b, 1 \leq q \leq Q_b \tag{7}$$

2. Solve the resulting system of equations: - For linear PDEs: Linear least squares (LLSQ) - For nonlinear PDEs: Nonlinear least squares with perturbations (NLSQ-perturb)

3. For time-dependent PDEs, employ block time-marching: - Divide the spatial-temporal domain into sequential time blocks - Solve each time block separately using the solution from the previous time block as the initial condition

3 Algorithm for Computing Optimal R_m

A major contribution of the paper is a novel algorithm to determine the optimal R_m value(s).

3.1 Residual Minimization Approach

The optimal R_m is computed by minimizing the residual norm of the least squares solution:

1. For Single-R_m-ELM, define the residual vector:

$$r(R_m) = \begin{bmatrix} \vdots \\ L(u^{LS}(x_p, R_m)) - f(x_p) \\ \vdots \\ B(u^{LS}(x_q, R_m)) - g(x_q) \\ \vdots \end{bmatrix} \tag{8}$$

2. Find R_{m0} that minimizes the norm of this residual:

$$R_{m0} = \arg\min_{R_m} \|r(R_m)\| \tag{9}$$

3. For Multi-R_m-ELM, find the vector $\mathbf{R_{m0}}$ that minimizes the residual norm:

$$\mathbf{R_{m0}} = \arg\min_{\mathbf{R_m}} \|r(\mathbf{R_m})\| \tag{10}$$

3.2 Differential Evolution Algorithm

The paper employs the differential evolution (DE) algorithm to solve these optimization problems:

1. Generate a population of candidate R_m values 2. Evaluate the cost function $\|r(R_m)\|$ for each candidate using Algorithm 1 or 2:

- Set hidden-layer coefficients using R_m (or $\mathbf{R_m}$)
- Evaluate neural network outputs on collocation points
- Solve linear/nonlinear system to get ω^{LS}
- Compute residual vector and its norm

3. Update the population using differential evolution operations 4. Repeat until convergence

This approach is a pre-processing procedure that only needs to be performed once for a given problem. The optimal R_m value remains effective across different resolutions and parameter configurations.

4 Implementation Improvements

A significant improvement to the ELM implementation is the use of forward-mode auto-differentiation to compute differential operators involving the output fields of the last hidden layer. This is implemented using the "ForwardAccumulator" in TensorFlow, replacing the reverse-mode auto-differentiation ("GradientTape") used in previous work.

This change dramatically reduces computational time because in ELM, the number of nodes in the last hidden layer is typically much larger than that of the input layer, making forward-mode differentiation more efficient.

5 Theoretical Findings

5.1 Properties of Optimal R_m in Single-R_m-ELM

Through extensive numerical experimentation, the paper establishes several important properties of the optimal R_m value:

- R_{m0} is largely independent of the number of collocation points
- R_{m0} has a stronger dependence on the number of training parameters for networks with a single hidden layer, but this dependence is weak for networks with two or more hidden layers
- R_{m0} generally decreases with increasing network depth, with a significant drop when moving from one to two hidden layers, and only slight decreases thereafter

- R_{m0} has only a weak dependence on the width of preceding hidden layers, tending to decrease slightly with increasing width

5.2 Properties of Optimal R_m in Multi-R_m-ELM

For Multi-R_m-ELM, the components of the optimal $\mathbf{R_{m0}}$ show more irregular relationships with simulation parameters, but still exhibit some patterns:

- Components of $\mathbf{R_{m0}}$ are largely independent of the number of collocation points
- The relationship between $\mathbf{R_{m0}}$ and the number of training parameters is less regular than in Single-R_m-ELM
- Using different R_m values for different hidden layers consistently produces more accurate results than using a single R_m value

6 Numerical Examples and Results

The paper presents extensive numerical experiments for both linear and nonlinear PDEs:

6.1 Function Approximation

For a 2D function approximation problem:

$$f(x,y) = \left(\frac{3}{2}\cos\left(\frac{3}{2}\pi x + \frac{9\pi}{20}\right) + 2\cos\left(\frac{3\pi x - \pi}{5}\right)\right)\left(\frac{3}{2}\cos\left(\frac{3}{2}\pi y + \frac{9\pi}{20}\right) + 2\cos\left(\frac{3\pi y - \pi}{5}\right)\right) \tag{11}$$

Key findings:

- ELM achieved accuracy with errors on the order of 10^{-8}
- Errors decreased exponentially with increasing number of collocation points and training parameters
- Multi-R_m-ELM consistently outperformed Single-R_m-ELM in accuracy

6.2 Poisson Equation

For the 2D Poisson equation:

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = f(x,y) \tag{12}$$

$$\text{with Dirichlet boundary conditions} \tag{13}$$

Results showed:

- ELM solution achieved maximum errors on the order of 10^{-8}
- Exponential convergence with respect to collocation points and training parameters
- ELM far outperformed classical FEM (2nd-order) in accuracy for the same computational cost
- ELM was competitive with high-order FEM, outperforming it for larger problem sizes

6.3 Nonlinear Helmholtz Equation

For the nonlinear Helmholtz equation:

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} - 100u + 10\cos(2u) = f(x, y) \tag{14}$$

$$\text{with Dirichlet boundary conditions} \tag{15}$$

Results demonstrated:

- Single-R_m-ELM achieved maximum errors on the order of 10^{-10}
- Multi-R_m-ELM consistently provided better accuracy
- ELM outperformed both classical and high-order FEM in terms of accuracy vs. computational cost

6.4 Viscous Burgers' Equation

For the time-dependent Burgers' equation:

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} - \nu\frac{\partial^2 u}{\partial x^2} = f(x, t) \tag{16}$$

$$\text{with appropriate boundary and initial conditions} \tag{17}$$

Results showed:

- ELM with block time-marching achieved maximum errors on the order of 10^{-9} across the entire space-time domain
- ELM dramatically outperformed both classical and high-order FEM for time-dependent problems
- As the temporal dimension increases (for longer-time simulations), ELM's advantage becomes even more prominent

7 Performance Comparison with FEM

The paper provides comprehensive comparisons between ELM and both classical and high-order FEM across multiple test problems:

7.1 Comparison with Classical FEM (2nd-order)

- ELM consistently and significantly outperforms classical FEM for all problems except for very small problem sizes
- For the same computational budget, ELM achieves orders of magnitude better accuracy
- For a target accuracy level, ELM requires substantially less computational time

7.2 Comparison with High-Order FEM

- For stationary PDEs, a crossover point exists with respect to problem size:
  - For smaller problem sizes: ELM and high-order FEM perform comparably, with high-order FEM slightly better
  - For larger problem sizes: ELM markedly outperforms high-order FEM
- For time-dependent PDEs, ELM with block time-marching consistently and significantly outperforms high-order FEM
- ELM produces superior performance for both h-type refinements (fixed element degree, varied mesh size) and p-type refinements (fixed mesh size, varied element degree)

8 Implications and Advantages

The paper demonstrates several important implications and advantages of the improved ELM approach:

8.1 Practical Benefits of Optimal R_m Computation

- The optimal R_m computation is a one-time pre-processing step that determines near-optimal values for subsequent calculations
- R_m values in a neighborhood of the optimal value lead to comparable accuracy, providing flexibility in implementation
- For time-dependent PDEs, the optimal R_m computed for the first time block can be used for all subsequent time blocks

8.2 Advantages of Multi-R_m-ELM

- Consistently produces more accurate results than Single-R_m-ELM under identical conditions
- Provides a systematic way to initialize different hidden layers with appropriate random weight magnitudes
- Particularly beneficial for deeper networks, where the optimal R_m values for different layers can vary significantly

8.3 Computational Performance

- The use of forward-mode auto-differentiation significantly reduces ELM training time compared to previous implementations
- ELM exhibits exponential convergence for smooth solutions, similar to high-order spectral/hp-finite element methods
- ELM is particularly advantageous for time-dependent problems, where its block time-marching approach outperforms traditional time-stepping schemes

8.4 Broader Implications for Scientific Computing

- The results provide strong evidence that neural network-based methods can be computationally competitive with traditional numerical techniques
- ELM's performance demonstrates that neural networks can excel in accuracy and computational efficiency for solving PDEs
- The approach represents a significant step toward making neural network-based methods practical and efficient for real-world scientific and engineering applications